EFFICIENCY Solutions

COMPUTER SCIENCE MENTORS 61A

October 26, 2026 – October 30, 2026

1 Efficiency

An order of growth (OOG) characterizes the runtime **efficiency** of a program as its input becomes extremely large. Since we care about rate of growth, we ignore constant coefficients and exclusively consider the fastest growing term. For example, on very large inputs, $2n^2 + 3n - 20$ behaves the same as n^2. Common runtimes, in increasing order of time, are: constant, logarithmic, linear, quadratic, and exponential.

Examples:

Constant time means that no matter the size of the input, the runtime of your program is consistent. In the function `f` below, no matter what you pass in for `n`, the runtime is the same.

```
def f(n):
    return 1 + 2
```

A common example of a linear OOG involves a single for/while loop. In the example below, as `n` gets larger, the amount of time to run the function grows proportionally.

```
def f(n):
    while n > 0:
        print(n)
        n -= 1
```

We can modify this while loop to get an example of logarithmic OOG. Suppose that, instead of subtracting 1 each time, we halve the size of `n`. For `n = 1000`, the program would take 10 iterations to terminate (since $2^{10} = 1024$). The runtime is propotional to $\log(n)$.

```
def f(n):
    while n > 0:
        print(n)
        n //= 2
```

An example of a quadratic runtime involves nested for loops. For every one of the `n` iterations of the outer loop, there is `n` work done in the inner loop. This means that the runtime is proportional to n^2.

```
def f(n):
    for i in range(n):
        for j in range(n):
            print(i*j)
```

Created by Ana Abell, Tien-Yu Hsu, Cadence Loh, and Rajoshi Basu. feedback form!
Provided by CSM 61A contributors under CC BY 4.0/3-clause BSD.

Exponential time: Assume `n` is a non-negative integer. Each non-base call below makes two recursive calls on an input smaller by one.

```
def branches(n):
    if n == 0:
        return
    branches(n - 1)
    branches(n - 1)
```

There is one call at depth 0, two at depth 1, four at depth 2, and 2^n at depth n. The total number of calls is $1 + 2 + 4 + \cdots + 2^n = 2^{n+1} - 1$, so the runtime is $\Theta(2^n)$. Count all calls, not just the depth of the recursion. Multiple recursive calls do not always imply exponential time: check whether each call continues branching or immediately reaches a base case.

1. What is the order of growth in time for the following functions? Give tight bounds in Θ notation. Assume `n` is a non-negative integer and arithmetic takes constant time. If different inputs follow different execution paths, describe their runtimes separately.

   (a)
   ```
   def strange_add(n):
       if n == 0:
           return 1
       else:
           return strange_add(n - 1) + strange_add(n - 1)
   ```

   $\Theta(2^n)$. Every non-base call makes two recursive calls. At depth k there are 2^k calls, for depths 0 through n. The total is $1 + 2 + \cdots + 2^n = 2^{n+1} - 1$ calls, with constant work per call.

   (b)
   ```
   def stranger_add(n):
       if n < 3:
           return n
       elif n % 3 ==  0:
           return stranger_add(n - 1) + stranger_add(n - 2) +
               stranger_add(n - 3)
       else:
           return n
   ```

   $\Theta(n)$ for positive multiples of 3; $\Theta(1)$ otherwise, including `n = 0`.

   Only calls whose argument is a positive multiple of 3 recurse. For example, `stranger_add(9)` calls the function on 8, 7, and 6. The calls on 8 and 7 return immediately through the **`else`** branch. The call on 6 calls the function on 5, 4, and 3; again, only the last call continues. The continuing chain is $9 \to 6 \to 3 \to 0$.

   Thus, each recursive stage does constant work and makes two constant-time calls plus one call on $n - 3$: $T(n) = T(n - 3) + \Theta(1)$. There are $n/3$ such stages, giving $\Theta(n)$ time. There are three calls in the code, but they do not form three continuing recursive branches. For inputs not divisible by 3, the initial call returns immediately.

2. Find the runtime for each of the following functions in Θ notation. Assume `n` is a non-negative integer and arithmetic and printing each take constant time.

`def mystery(n):`
`  total = 0`
`  for i in range(n // 2):`
`  j = 1`
`  while j < n:`
`  j *= 2`
`  total += i * j`
`  return total`	

`def foo(n):`
`  if n == 0:`
`  return`
`  for i in range(n // 2):`
`  print(i)`
`  foo(n // 2)`	

- `mystery`: $\Theta(n \log n)$. The outer loop runs $\lfloor n/2 \rfloor$ times. On every iteration, `j` resets to 1 and doubles until it reaches or exceeds n, so the inner loop takes $\Theta(\log n)$ time. Multiply the two counts.
- `foo`: $\Theta(n)$. The work across successive calls is proportional to $n/2 + n/4 + n/8 + \cdots$, which sums to $\Theta(n)$. The recursion has logarithmic depth, but its loops get shorter at each level; multiplying n by the depth would overcount the work.

3. **Fast Exponentiation:** in this problem, we will examine a real-world algorithm used to improve the speed of calculating exponents. You can assume for the purposes of this problem that multiplication is a constant time operation.

   (a) First, determine the runtime efficiency of the naive exponentiation algorithm.

```
def exp(b, n):
    if n == 0:
        return 1
    else:
        return b * exp(b, n - 1)
```

   Linear runtime. n decreases by 1 each call, so there are naturally n calls.

(b) Now, express the runtime of the fast exponentiation algorithm.

```
def fast_exp(b, n):
    if n == 0:
        return 1
    elif n % 2 == 0: # Assume square runs in constant time
        return square(fast_exp(b, n // 2))
    else:
        return b * fast_exp(b, n - 1)
```

$\theta(\log n)$. n is halved each call, so the number of calls is the number of times n must be halved to get to 1. This is $\log n$, or logarithmic.

(c) What about this slightly modified version of `fast_exp`?

```
def fast_exp(b, n):
    for _ in range(50 * n):
        print("Killing time")
    if n == 0:
        return 1
    elif n % 2 == 0:
        return square(fast_exp(b, n // 2))
    else:
        return b * fast_exp(b, n - 1)
```

$\theta(n)$. Ignore the constant term. The first call will perform n operations, the second call will perform $n/2$ operations, the third will perform $n/4$ operations, etc. Using geometric series, we see this adds up to $2n$, which is n (linear) if we ignore constant terms.

2 Optional Midterm Review

4. [Exam Level] Implement the `Poll` class and the `tally` function, which takes a choice `c` and returns a list describing the number of votes for `c`. This list contains pairs, each with a name and the number of times `vote` was called on that `choice` at the `Poll` with that name. Pairs can be in any order. Assume all `Poll` instances have distinct names. Hint: the dictionary `get(key, default)` method (MT 2 guide, page 1 top-right) returns the value for a `key` if it appears in the dictionary and `default` otherwise. [Adapted from CS61A Fa18 Midterm 2 Q5(a)]

```
class Poll:
    s = []

    def __init__(self, n):

        self.name = ____________________________________________________

        self.votes = {}

        ____________________________________________________________________

    def vote(self, choice):

        self._____________________________ = _____________________________________


def tally(c):
    """Tally all votes for a choice c as a list of (poll name, vote count)
       pairs.
    >>> a, b, c = Poll('A'), Poll('B'), Poll('C')
    >>> c.vote('dog')
    >>> a.vote('dog')
    >>> a.vote('cat')
    >>> b.vote('cat')
    >>> a.vote('dog')
    >>> tally('dog')
    [('A', 2), ('C', 1)]
    >>> tally('cat')
    [('A', 1), ('B', 1)]
    """

    return ____________________________________________________________
```

```
class Poll:
    s = []
    def __init__(self, n):
        self.name = n
        self.votes = {}
        Poll.s.append(self)
    def vote(self, choice):
        self.votes[choice] = self.votes.get(choice, 0) + 1

def tally(c):
    return [(p.name, p.votes[c]) for p in Poll.s if c in p.votes]
```

5. Given a list of integers `lst`, return the maximum sum of a subset of size `n`. If `n` is greater than or equal to the length of `lst`, just return the sum of the elements `lst`.

```
def max_subset_sum(lst, n):
    """
    >>> max_subset_sum([1, 2, 3, 4], 2)
    7
    >>> max_subset_sum([1, 4, 2, 0, 6], 3)
    12
    """

    if _____________________________:

        _____________________________

    elif _____________________________:

        _____________________________

    with_elem = _____________________________ + __________

    without_elem = _____________________________________

    return _____________________________________________
```

```
def max_subset_sum(lst, n):
    if n == 0:
        return 0
    elif len(lst) <= n:
        return sum(lst)
    with_elem = max_subset_sum(lst[1:], n - 1) + lst[0]
    without_elem = max_subset_sum(lst[1:], n)
    return max(with_elem, without_elem)
```

6. What Would Python Display?

```
class SkipMachine:
    skip = 1
    def __init__(self, n=2):
        self.skip = n + SkipMachine.skip

    def generate(self):
        current = SkipMachine.skip
        while True:
            yield current
            current += self.skip
            SkipMachine.skip += 1

p = SkipMachine()
twos = p.generate()
SkipMachine.skip += 1
twos2 = p.generate()
threes = SkipMachine(3).generate()
```

(a) `next(twos)`

2

(b) `next(threes)`

2

(c) `next(twos)`

5

(d) `next(twos)`

8

(e) `next(threes)`

7

(f) `next(twos2)`

5